

Frontend Integration Guide - Step 2: Faculty Allocation & Vacancy Identification

This document provides a technical roadmap for the frontend developer to implement **Step 2 (Faculty Allocation & Vacancy Identification)** of the CHB Recruitment Portal.

Overview

Step 2 converts Step 1 requirement numbers into a practical, approval-ready vacancy decision.

The system combines:

- existing faculty roster and effectiveness,
- qualification/age/workload risk checks,
- AI-assisted recommendation logic.

Outcome: The frontend should present a **recommended vacancy count with clear justification**, highlight underutilized/overloaded areas, and enforce anomaly acknowledgment before final confirmation.

Backend Module Snapshot

- **Backend Prefix:** `/api/vacancies`
 - **Roles:**
 - **PRINCIPAL:** full Step 2 actions (add/update/delete faculty, suggest AI vacancy, acknowledge anomalies, confirm vacancy)
 - **ADMIN:** read-only access to faculty list and assessment
-

Core Functional Blocks

1. Existing Faculty Capture (Input Quality for AI)

Before recommendation, the principal must maintain accurate faculty records for each institution/course/year.

- **Key Endpoint(s):**
 - `POST /api/vacancies/faculty`
 - `PUT /api/vacancies/faculty/{faculty_id}`
 - `GET /api/vacancies/faculty?institution_id={id}&course_id={id}&academic_year=2026-2027&skip=0&limit=20`
 - `DELETE /api/vacancies/faculty/{faculty_id}?reason=...`
- **Request body (create):**

```
{
  "institution_id": 1,
  "course_id": 101,
  "employee_id": "EMP-CE-001",
```

```

"full_name": "Dr. Asha Kulkarni",
"designation": "Assistant Professor",
"employment_type": "PERMANENT",
"qualification": "M.E.",
"specialization": "Computer Engineering",
"date_of_birth": "1988-03-12",
"date_of_joining": "2019-07-01",
"status": "ACTIVE",
"academic_year": "2026-2027",
"qualifications": [
  {
    "degree": "M.E.",
    "specialization": "Computer Engineering",
    "university": "SPPU",
    "year_of_passing": 2012,
    "is_highest": true
  }
]
}

```

- **Important frontend validations:**

- `date_of_joining` must not be future date.
- Prevent duplicate `employee_id` per institution + year.
- Keep `academic_year` format consistent (YYYY-YYYY).

Note: Add/update/delete operations reset related vacancy assessment to **DRAFT**, so UI should prompt user to regenerate suggestion.

2. Vacancy Suggestion + AI Analysis

This is the core Step 2 operation where backend computes system vacancy and returns AI analysis.

- **Key Endpoint(s):**

- `POST /api/vacancies/suggest`
- `POST /api/vacancies/ai-analysis` (same behavior; can be treated as alias)

- **Request body:**

```

{
  "institution_id": 1,
  "course_id": 101,
  "academic_year": "2026-2027"
}

```

- **Expected response shape:**

```

{
  "status": "success",

```

```

"data": {
  "system_vacancy": 4,
  "ai_analysis": {
    "ai_suggested_vacancy": 5,
    "justification": "Faculty with age/qualification constraints were
excluded...",
    "risk_level": "MEDIUM",
    "overloaded": ["Core labs require additional support"],
    "underutilized": ["Theory section has surplus coverage"]
  }
}

```

3. Assessment Read API (Screen Reload / Detail View)

Use this endpoint to fetch persisted Step 2 state after page reload or navigation.

- **Endpoint:**

- `GET /api/vacancies/assessment?institution_id={id}&course_id={id}&academic_year=2026-2027`

- **Returns:**

- required/effective counts,
- suggested and confirmed vacancy,
- assessment status (`DRAFT`, `AI_SUGGESTED`, `CONFIRMED`),
- anomaly list,
- `unacknowledged_high_count` guard value.

4. Anomaly Acknowledgment Workflow

High-severity anomalies must be acknowledged before confirmation.

- **Endpoint:**

- `POST /api/vacancies/anomalies/{anomaly_id}/acknowledge`

- **Request body:**

```

{
  "remarks": "Candidate replacement planned; qualification gap acknowledged."
}

```

5. Final Vacancy Confirmation

Once all required anomalies are acknowledged, principal confirms final vacancy.

- **Endpoint:**

- `POST /api/vacancies/confirm?institution_id=1&course_id=101&academic_year=2026-2027`

- **Request body:**

```
{
  "confirmed_vacancy": 5
}
```

- **Backend guardrails to reflect in UI:**

- If already confirmed: edits blocked.
- If HIGH anomalies unacknowledged: confirmation blocked.
- Large deviation from suggested vacancy is audit-logged; UI should request remarks/comment in UX even if not mandatory in API.

Suggested Frontend UX Flow

1. Select Institution + Course + Academic Year.
 2. Load faculty list (`GET /faculty`) and existing assessment (`GET /assessment`).
 3. Allow principal to add/edit/remove faculty entries.
 4. Trigger `Suggest Vacancy` (`POST /suggest` or `/ai-analysis`).
 5. Show recommendation summary cards:
 - Required Faculty
 - Effective Existing Faculty
 - System Vacancy
 - AI Suggested Vacancy
 6. Render AI justification block and workload flags:
 - Overloaded areas
 - Underutilized areas
 7. Show anomaly table with severity chips and acknowledge action.
 8. Enable `Confirm Vacancy` only when `unacknowledged_high_count == 0`.
 9. After confirmation, lock editing and show status `CONFIRMED`.
-

UI Checklist for Step 2

A. Faculty Allocation Panel

- Table with status and effectiveness indicator (`is_effective`).
- Form drawer/modal for create/update faculty with qualification list.
- Soft-delete action with mandatory reason prompt.

B. Vacancy Intelligence Panel

- KPI cards for counts (`required_count`, `total_existing`, `effective_existing`, `suggested_vacancy`).
- AI recommendation card:

- `ai_suggested_vacancy`
- `justification`
- risk label (if present)
- Two lists:
 - `overloaded`
 - `underutilized`

C. Anomaly & Approval Panel

- Filterable anomaly table by severity.
- Per-anomaly acknowledge button + remarks modal.
- Confirmation section with editable `confirmed_vacancy`.
- Hard disable confirm button when HIGH anomalies remain unacknowledged.

API Envelope & Error Handling

- Success envelope:

```
{
  "status": "success",
  "data": {}
}
```

- Error envelope:

```
{
  "status": "error",
  "code": "HTTP_ERROR",
  "message": "All HIGH severity anomalies must be acknowledged before confirmation"
}
```

Frontend should always surface `message` directly for actionable feedback.

Common Step 2 errors to map:

- `400`: Step 1 not completed (intake/requirement missing), validation, confirmation gates.
- `403`: assessment already confirmed and cannot be edited.
- `404`: faculty/assessment/anomaly not found.
- `422`: query parameter format errors (e.g., non-integer `institution_id`, `course_id`).

Dependency on Step 1

Step 2 assumes Step 1 exists for the same `course_id` + `academic_year`.

If missing:

- faculty can still be managed,
- but vacancy suggestion should show backend error:
 - **Step 1 Intake not defined for this Course and year**
 - or **Faculty Requirement not generated yet (Step 1)**

UI should detect and guide user to complete Step 1 first.

Recommended Frontend States

- **DRAFT**: no final suggestion/needs regeneration.
- **AI_SUGGESTED**: recommendation available, awaiting final confirmation.
- **CONFIRMED**: read-only summary for approval trail and downstream Step 3.

Postman Reference

Refer to the **Vacancy Identification (Step 2)** requests in **CHB_Portal.postman_collection.json** for live request/response examples aligned with these APIs.